

BEP-007: Tuplas e Dicionários

Conheça outras estruturas de dados fundamentais em Python

Exercícios - Dicionários

Exercício 1: Agenda Telefônica



```
# Crie uma agenda telefônica usando dicionários
# Adicione, busque e atualize contatos

# Solução:
agenda = {}

# Adicionando contatos
agenda["João"] = {"telefone": "99999-1111", "email": "joao@email.com"}
agenda["Maria"] = {"telefone": "99999-2222", "email": "maria@email.com"}
agenda["Pedro"] = {"telefone": "99999-3333", "email": "pedro@email.com"}

print("Agenda:", agenda)

# Buscando contato
nome_busca = "Maria"
if nome_busca in agenda:
    contato = agenda[nome_busca]
    print(f"{nome_busca}: {contato['telefone']}")
else:
    print(f"Contato {nome_busca} não encontrado")

# Atualizando contato
agenda["João"]["telefone"] = "99999-1122"
print(f"Telefone do João atualizado: {agenda['João']['telefone']}")

# Listando todos os contatos
for nome, dados in agenda.items():
    print(f"{nome}: {dados['telefone']} - {dados['email']}")
```

Exercício 2: Sistema de Notas

```
# Crie um sistema de notas usando dicionários
# Calcule médias e determine status dos alunos

# Solução:
notas_alunos = {
```

```
"Ana": [8.5, 7.0, 9.2],
"Bruno": [6.8, 8.0, 7.5],
"Carla": [9.0, 8.5, 9.5],
"Diego": [7.2, 6.5, 8.0]
}

print("Sistema de Notas:")
for aluno, notas in notas_alunos.items():
    media = sum(notas) / len(notas)
    status = "Aprovado" if media >= 7.0 else "Reprovado"

    print(f"{aluno}:")
    print(f"  Notas: {notas}")
    print(f"  Média: {media:.2f}")
    print(f"  Status: {status}")

# Encontrando melhor aluno
melhor_aluno = max(notas_alunos, key=lambda x: sum(notas_alunos[x])/len(notas_alunos[x]))
print(f"Melhor aluno: {melhor_aluno}")

# Adicionando nova nota
notas_alunos["Ana"].append(8.8)
print(f"Ana com nova nota: {notas_alunos['Ana']}")
```

Exercício 3: Inventário de Produtos



```
# Crie um sistema de inventário com dicionários
# Gerencie produtos, preços e estoque

# Solução:
inventario = {
    "notebook": {"preco": 2500.00, "estoque": 10, "categoria": "Eletrônicos"},
    "mouse": {"preco": 50.00, "estoque": 25, "categoria": "Acessórios"},
    "teclado": {"preco": 120.00, "estoque": 15, "categoria": "Acessórios"}
}

# Calculando valor total do inventário
valor_total = 0
for produto, dados in inventario.items():
    valor_produto = dados["preco"] * dados["estoque"]
    valor_total += valor_produto

    print(f"{produto.title()}:")
    print(f"  Preço: R$ {dados['preco']:.2f}")
    print(f"  Estoque: {dados['estoque']} unidades")
    print(f"  Valor total: R$ {valor_produto:.2f}")

print(f"Valor total do inventário: R$ {valor_total:.2f}")

# Produtos por categoria
categorias = {}
for produto, dados in inventario.items():
    categoria = dados["categoria"]
    if categoria not in categorias:
        categorias[categoria] = []
    categorias[categoria].append(produto)
```

```
print("Produtos por categoria:", categorias)
```



Exercício 4: Sistema de Vendas



```
# Crie um sistema de vendas mensais
# Analise dados e gere relatórios

# Solução:
vendas = {
    "janeiro": 15000,
    "fevereiro": 18000,
    "marco": 12000,
    "abril": 22000,
    "maio": 19000,
    "junho": 21000
}

# Estatísticas básicas
total_vendas = sum(vendas.values())
media_vendas = total_vendas / len(vendas)
maior_venda = max(vendas.values())
menor_venda = min(vendas.values())

print(f"Total de vendas: R$ {total_vendas}")
print(f"Média mensal: R$ {media_vendas:.2f}")
print(f"Maior venda: R$ {maior_venda}")
print(f"Menor venda: R$ {menor_venda}")

# Mês com maior venda
mes_maior_venda = max(vendas, key=vendas.get)
print(f"Mês com maior venda: {mes_maior_venda} (R$ {vendas[mes_maior_venda]})")

# Crescimento mensal
meses = list(vendas.keys())
for i in range(1, len(meses)):
    mes_atual = meses[i]
    mes_anterior = meses[i-1]
    crescimento = vendas[mes_atual] - vendas[mes_anterior]
    print(f"Crescimento de {mes_anterior} para {mes_atual}: R$ {crescimento}")
```



Exercício 5: Sistema de Login



```
# Crie um sistema de login com usuários e senhas
# Implemente autenticação e níveis de acesso

# Solução:
usuarios = {
    "admin": {"senha": "123456", "nivel": "administrador", "ativo": True},
    "usuario1": {"senha": "senha123", "nivel": "usuario", "ativo": True},
    "usuario2": {"senha": "minhasenha", "nivel": "usuario", "ativo": False}
}
```

```
def verificar_login(usuario, senha):
    if usuario not in usuarios:
        return False, "Usuário não encontrado"

    dados = usuarios[usuario]
    if dados["senha"] == senha and dados["ativo"]:
        return True, dados["nivel"]
    elif not dados["ativo"]:
        return False, "Usuário inativo"
    else:
        return False, "Senha incorreta"

# Testando o sistema
testes = [
    ("admin", "123456"),
    ("usuario1", "senha_errada"),
    ("usuario2", "minhasenha"),
    ("inexistente", "qualquer")
]

for usuario, senha in testes:
    resultado, info = verificar_login(usuario, senha)
    status = "Sucesso" if resultado else "Falha"
    print(f>Login '{usuario}': {status} - {info}")
```



Exemplo: Sistema de Configuração



```
# Sistema de configuração de aplicação
config = {
    "database": {
        "host": "localhost",
        "port": 5432,
        "name": "meuapp"
    },
    "server": {
        "host": "0.0.0.0",
        "port": 8080,
        "debug": True
    },
    "logging": {
        "level": "INFO",
        "file": "app.log"
    }
}

# Acessando configurações
db_config = config["database"]
print(f"Banco: {db_config['host']}:{db_config['port']}")
print(f"Debug: {config['server']['debug']}")

# Atualizando configurações
config["server"]["debug"] = False
config["database"]["port"] = 3306

print("Configuração atualizada:")
for secao, configuracoes in config.items():
```

```
print(f"{secao}:")  
for chave, valor in configuracoes.items():  
    print(f"  {chave}: {valor}")
```

⚠ Dicas para os Exercícios

- **Acesso seguro:** Use `get()` para evitar erros
- **Verificação:** Use `in` antes de acessar
- **Iteração:** Use `items()` para chave-valor
- **Funções:** Aproveite `max()`, `min()` com `key=`

